Production Honesty Ladder

their_production stays false until a recorded third-party exclusive weld. Museum is not production.

Source: gate/production_skin.py

Overview

Production skin - honesty gate + readiness ladder.

their_production: third-party (or env) production weld only.
dogfood_weld: first-party recorded weld - lifts proof/deploy, not their_production.

CANONICAL: Own permission on irreversible acts for any power that needs it - scarcity is the DENY, not the narrative.

CHECKLIST

- {'id': 'act_door', 'label': 'POST /v1/act welded closed world exists'}
- {'id': 'demo_hop', 'label': 'Public demo hop fails closed with verify_url'}
- {'id': 'charge_only', 'label': 'CHARGE-only DEAD->LIVE (license fuse + refuse list)'}
- {'id': 'pii_wall', 'label': 'PAS paths reject PII'}
- {'id': 'register', 'label': 'Register + operator weld checkout live'}
- {'id': 'action_os', 'label': 'Action OS formula published'}
- {'id': 'proof_suite', 'label': 'Proof suite all_pass'}
- {'id': 'stranger_verify', 'label': 'Stranger verify via Velaru'}

INVENTOR: Nisaba LLC / Gate

SPEC: gate-production-skin-v2

Source appendix

```
"""Production skin  -  honesty gate + readiness ladder.

their_production: third-party (or env) production weld only.
dogfood_weld: first-party recorded weld  -  lifts proof/deploy, not their_production.
"""
from __future__ import annotations

import os
from datetime import datetime, timezone
from typing import Any

SPEC = "gate-production-skin-v2"
INVENTOR = "Nisaba LLC / Gate"

CANONICAL = (
    "Own permission on irreversible acts for any power that needs it  -  "
    "scarcity is the DENY, not the narrative."
)

CHECKLIST = (
    {"id": "act_door", "label": "POST /v1/act welded closed world exists"},
```

```
    {"id": "demo_hop", "label": "Public demo hop fails closed with verify_url"},
    {"id": "charge_only", "label": "CHARGE-only DEAD->LIVE (license fuse + refuse list)"},
    {"id": "pii_wall", "label": "PAS paths reject PII"},
    {"id": "register", "label": "Register + operator weld checkout live"},
    {"id": "action_os", "label": "Action OS formula published"},
    {"id": "proof_suite", "label": "Proof suite all_pass"},
    {"id": "stranger_verify", "label": "Stranger verify via Velaru"},
)


def _now() -> str:
    return datetime.now(timezone.utc).isoformat()


def their_production() -> bool:
    """True only when a real exclusive third-party production weld is recorded.

    GATE_PRODUCTION_WELDED alone is insufficient  -  also requires
    GATE_PRODUCTION_EXCLUSIVE=1 (ops honesty for emergency env flips) OR a DB
    row with exclusivity_attested + exclusive_door_url.
    """
    flag = os.getenv("GATE_PRODUCTION_WELDED", "").strip().lower()
    exclusive_env = os.getenv("GATE_PRODUCTION_EXCLUSIVE", "").strip().lower() in (
        "1",
        "true",
        "yes",
        "on",
    )
    if flag in ("1", "true", "yes", "on") and exclusive_env:
        return True
    try:
        from gate import db as gate_db
    except ImportError:
        import db as gate_db  # type: ignore[no-redef]
    checker = getattr(gate_db, "has_gate_production_weld", None)
    if callable(checker):
        return bool(checker())
    return False


def has_dogfood_weld() -> bool:
    try:
        from gate import db as gate_db
    except ImportError:
        import db as gate_db  # type: ignore[no-redef]
    checker = getattr(gate_db, "has_dogfood_weld", None)
    if callable(checker):
        return bool(checker())
    return False


def record_dogfood_weld(
    *,
    write_path: str,
    operator: str,
    note: str = "",
) -> dict[str, Any]:
```

```
    """Record first-party dogfood weld. Does NOT flip their_production."""
    try:
        from gate import db as gate_db
    except ImportError:
        import db as gate_db  # type: ignore[no-redef]
    fn = getattr(gate_db, "record_dogfood_weld", None)
    if not callable(fn):
        return {"ok": False, "error": "db.record_dogfood_weld unavailable"}
    row = fn(write_path=write_path, operator=operator, note=note)
    return {"ok": True, "dogfood": True, "their_production": False, "weld": row}


def record_production_weld(
    *,
    write_path: str,
    counterparty: str,
    note: str = "",
    stripe_session_id: str | None = None,
    confirm: bool = False,
    exclusive_door_url: str | None = None,
    door_kind: str | None = None,
    worker_fingerprint: str | None = None,
) -> dict[str, Any]:
    """Record third-party production weld. Requires confirm + exclusivity attestation."""
    if not confirm:
        return {
            "ok": False,
            "error": "confirm=True required  -  their_production only for a real third-party write
  ",
            "their_production": their_production(),
        }
    try:
        from gate import db as gate_db
    except ImportError:
        import db as gate_db  # type: ignore[no-redef]
    fn = getattr(gate_db, "record_production_weld", None)
    if not callable(fn):
        return {"ok": False, "error": "db.record_production_weld unavailable"}
    try:
        row = fn(
            write_path=write_path,
            counterparty=counterparty,
            note=note,
            stripe_session_id=stripe_session_id,
            exclusive_door_url=exclusive_door_url,
            door_kind=door_kind,
            worker_fingerprint=worker_fingerprint,
        )
    except ValueError as exc:
        return {"ok": False, "error": str(exc), "their_production": their_production()}
    return {"ok": True, "dogfood": False, "their_production": True, "weld": row}


def checklist_status(proof: dict[str, Any] | None = None) -> list[dict[str, Any]]:
    """Auto-evaluate readiness checklist from a proof suite manifest."""
    proof = proof or {}
    inv_ids = {i["id"] for i in proof.get("invariants") or [] if i.get("passes")}
```

```
    out = []
    mapping = {
        "act_door": "exclusive_door",
        "demo_hop": "dead_holds",
        "charge_only": "license_fuse_rejects_empty_charge",
        "pii_wall": "pii_rejected",
        "register": "register_not_saas",
        "action_os": "formula_present",
        "proof_suite": None,  # special
        "stranger_verify": "dead_holds",
    }
    for item in CHECKLIST:
        iid = item["id"]
        if iid == "proof_suite":
            ok = bool(proof.get("all_pass"))
        else:
            ok = mapping.get(iid) in inv_ids
        out.append({**item, "passes": ok})
    return out


def manifest(public_url: str) -> dict[str, Any]:
    base = (public_url or "").rstrip("/")
    prod = their_production()
    dogfood = has_dogfood_weld()
    try:
        from gate import proof_suite as proof_mod
    except ImportError:
        import proof_suite as proof_mod  # type: ignore[no-redef]

    proof = proof_mod.manifest(base)
    ready = proof.get("readiness") or {}
    checks = checklist_status(proof)
    checks_pass = sum(1 for c in checks if c["passes"])
    return {
        "spec": SPEC,
        "name": "Production skin",
        "inventor": INVENTOR,
        "evaluated_at": _now(),
        "canonical": CANONICAL,
        "their_production": prod,
        "dogfood_weld": dogfood,
        "readiness": ready,
        "checklist": checks,
        "checklist_pass": checks_pass,
        "checklist_total": len(checks),
        "shipped_now": [
            "POST /v1/act * POST /demo/hop",
            "CHARGE-only resurrection doctrine",
            "Register + operator weld checkout",
            "Action OS formula + family map",
            "Expanded proof suite + readiness ladder",
            "Stranger verify via Velaru",
            "License fuse * restraint * settlement windows",
        ],
        "not_yet": [
            *(
```

```
                []
                if prod
                else ["Third-party production weld on their write (their_production)"]
            ),
            *([] if dogfood else ["First-party dogfood weld record"]),
            "Force/battlefield door (category only)",
        ],
        "force_production_weld": False,
        "links": {
            "scorecard": f"{base}/.well-known/scorecard.json",
            "proof": f"{base}/.well-known/proof-suite.json",
            "runbook": f"{base}/runbook",
            "action_os": f"{base}/.well-known/action-os.json",
            "register": f"{base}/register",
            "operator": f"{base}/operator",
            "dogfood": f"{base}/dogfood",
            "production_weld": f"{base}/production-weld",
        },
        "page": f"{base}/production-skin",
        "gatekeep": "Proof lifts deploy. Weld flips production. Ours.",
    }
```